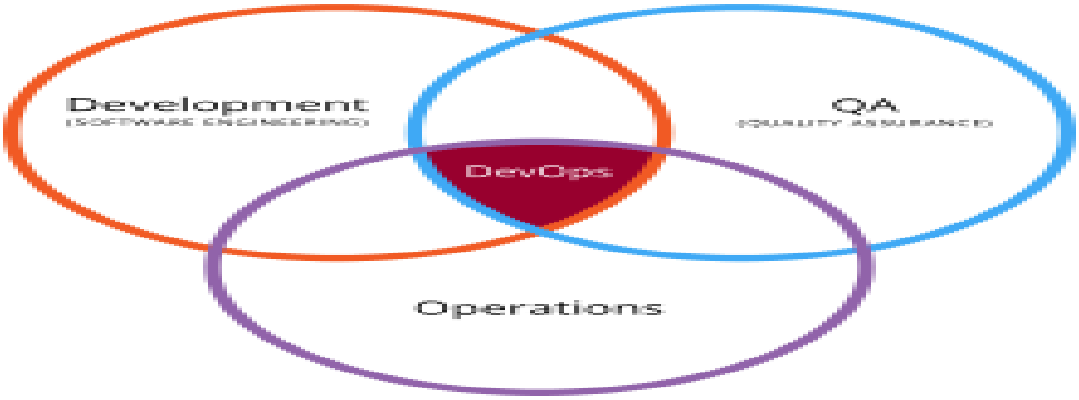
Week 9 Development (Session 1)

9. 1.1Introduction

- DevOps is a software development process that emphasizes communication and collaboration between product management, software development, and operations professionals.
- DevOps also automates the process of software integration, testing, deployment and infrastructure changes.
- It aims to establish a culture and environment where building, testing, and releasing software can happen rapidly, frequently, and more reliably.
- The end goal of DEVOPS is to reduce "time to market" and improve the quality of the released product.

9. 1.2 Overview of DevOps



- Venn diagram showing DevOps as the intersection of development (software engineering), operations and quality assurance (QA)
- DevOps promotes a set of processes and methods for thinking about communication and collaboration between departments of development, QA (quality assurance), and IT operations.

9.1.2.1 DevOps toolchain

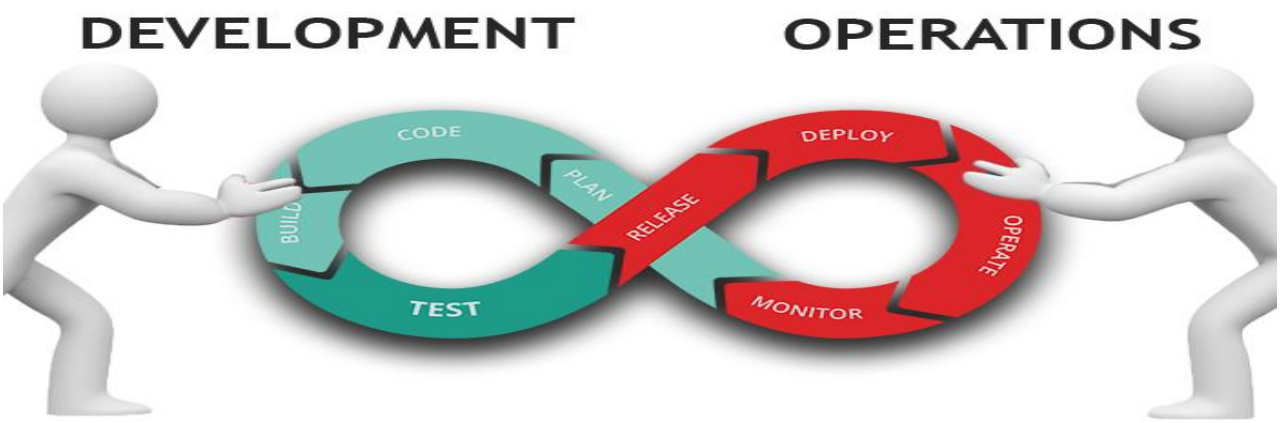


Illustration showing stages in a DevOps toolchain

- DevOps focuses on cross-functional mode of working. DevOps tool set consists of multiple tools. These tools fit into one or more categories starting from development to delivery process.
- Code — Code development and review, version control tools, code merging.
- Build — Continuous integration tools, build status.
- Test —Continuous testing tools that provide feedback on business risks.
- Package — Artifact repository, application pre-deployment staging.
- Release — Change management, release approvals, release automation.
- Configure — Infrastructure configuration and management, Infrastructure as Code tools.

9.1.3.Working Principles of Devops

- DevOps is a mindset or a philosophy that encompasses collaboration, communication, sharing, openness, and a holistic approach to software development.
- DevOps relies on a comprehensive set of strategies and methodologies. They ensure the timely delivery of quality software. DevOps principles guide how to organize a DevOps environment.

9.1.3.Nine principles of Devops

- Incremental Releases
- Automation
- DevOps Pipeline
- Continuous Integration
- Continuous Delivery
- Continuous Monitoring
- Feedback Sharing
- Version Control
- Collaboration

9.1.3.1 Incremental Releases

- Previously developers in Dev Ops teams wasted time in deploying the code. This delay in code release caused delayed progress.
- When developers have to wait for a prolonged period of time to contribute code, errors, and incompatibility issues.
- In a DevOps environment, a single project is divided into small, manageable chunks, and teams submit their code in increments. That makes it easier to troubleshoot problematic code before it gets released to

production. Depending on the workflow, DevOps teams release code updates and bug fixes on a daily, weekly, or monthly basis.

- Incremental releases make the development and deployment cycle more flexible. As a result, teams can quickly respond to sudden changes and fix errors and bugs immediately.

9.1.3.2 Automation

- One of the major practices of DevOps is automating the software development process as much as possible. Automating workflows allows developers to focus solely on writing code and developing new features.
- Anything that can be automated is automated in a DevOps environment.
- **For example**, instead of wasting time on manually checking code for errors, DevOps teams use different software solutions to build and test applications automatically. It is as simple as running a single command to compile the source code to determine if it will work in production.
- If the application is written in a language that doesn't need to be compiled then DevOps teams can run automated tests to check if the new code is production-ready. If there are any errors or bugs in the code, automation will trigger an alert letting developers know which lines of code are causing issues.
- Automation also plays a vital role in dealing with infrastructure management procedures.
- In a DevOps environment, teams utilize Infrastructure-as-Code (IaC). Infrastructure-automation software, such as Pulumi, helps manage the provisioning and decommissioning of resources.
- The goal is to enable developers to quickly provision anything from containers and virtual machines to bare metal cloud servers, storage, databases, and other infrastructure.
- Infrastructure-as-Code (IaC) made it possible for enterprises to deploy hundreds of servers or applications every day and destroy them when they are not needed. By letting automation take care of infrastructure management procedures, teams can focus their efforts on building and deploying software.

9.1.3.3 DevOps Pipeline

- DevOps aims to establish a repeatable system, a loop that facilitates continuity in development. To achieve that, DevOps teams create pipelines.
- A pipeline denotes a repeatable system made up of stages through which code has to pass before being deployed to production. A typical DevOps pipeline consists of four primary phases:
- Develop. First, the developers have to write the code.
- Build. Then the team compiles the code into a build to check for errors.
- Test. After the build phase, operations teams run tests to ensure that the new code will behave as intended in the production environment.
- Deploy. Once the new code has passed the testing phase, it gets deployed to the end-user.

- DevOps teams implement critical strategies to achieve a steady flow of code through the pipeline. The most important among them is Continuous Integration (CI) and Continuous Delivery (CD), also known as CI/CD.
- **Continuous Integration** has to do with enabling multiple developers to submit and merge their code regularly, while **Continuous Delivery** is all about releasing code updates to production as often as possible.

9.1. 3.4 Continuous Integration

- Continuous Integration (CI) plays a important role in a DevOps pipeline. It encourages developers to submit their code to a central code repository multiple times a day.
- Integration of smaller chunks of code regularly prevents bad code from moving down the pipeline and also service interruptions is significantly reduced. Some of the largest organizations that implement DevOps practices commit new code hundreds of times a day.
- Another critical aspect of Continuous Integration is automated testing. Before developers commit their code to the master branch, they create builds of the project to determine if the new code is compatible with the existing code.
- If the build is successful, developers will submit their code to the shared repository. CI relies on a version control system, a central code repository that helps teams track code changes and manage merge requests.

9.1.3.5 Continuous Delivery

- Continuous Delivery (CD) is all about releasing code updates frequently and fast. CD relies on developers to manually deploy code to production as opposed to Continuous Deployment, which takes an automated approach to code release.
- Developers working in a CD environment need to keep in mind that their code contributions may get deployed to production at any moment. Having passed all tests and reviews, code updates can be released to production with a click of a button.
- Numerous benefits come with continuous delivery. First, it eliminates the risk of downtime and other performance issues because code changes are usually minor. Second, CD enables organizations to release high-quality features much easier, ensuring faster time-to-market, and ultimately eliminating fixed costs associated with the deployment process.
- Quality Assurance teams set a committed code test using automation testing tools such as UFT, Ranorex, or Selenium. If QA finds bugs, they go back to the engineers. This stage features version control to look for any integration problem in advance. The Version Control System allows developers to record the changes in the files for sharing with other team members, regardless of location.
- Codes that pass automated tests get integrated into one shared repository. Several code submissions serve to prevent future drastic differences in code branch and mainline code, lengthening integration. Popular continuous integration tools include Git, Jenkins, TeamCity, and Bamboo.
- DevOps tools help engineers deploy a product in increments, such as Puppet, Chef, Google Cloud Deployment Manager, and Azure Resource Manager.

9.1.3.6 Continuous Monitoring

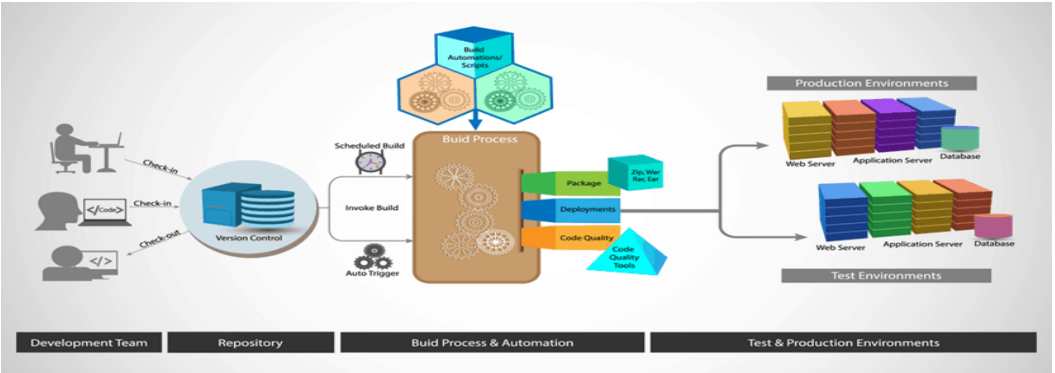
- Continuous Monitoring builds on the concepts of CI/CD, and it ensures the application performs without issues. DevOps teams implement monitoring technologies and techniques to keep track of how the application is behaving.
- DevOps teams monitor logs, apps, systems, and infrastructure.Once a problem is detected, DevOps teams can quickly revert the app to a previous state. During that time, the team works on resolving known issues without making the user aware that the code is updated continuously.
- Continuous monitoring also helps DevOps teams detect issues that are hindering productivity in the pipeline. After each release cycle, teams should optimize the CI/CD pipeline to eliminate any bottlenecks and ensure a smooth transition of code from one stage to the next.

9.1.3.7 Feedback Sharing

- DevOps thrives on feedback. Developers need actionable information from different sources to help them improve the overall quality of the application. Without feedback, DevOps teams can fall victim to spending time on building products that don't bring value to stakeholders or customers.
- DevOps teams usually gather feedback from stakeholders, end-users, and software-based monitoring technologies before, during, and after the release cycle. They collect feedback through various channels, such as social media and surveys, or by discussing with colleagues.
- Teams need to have procedures that will help them shift through the feedback they've gathered to extract the most valuable information. Misleading or inaccurate feedback could prove to be detrimental to the entire development process.

9.1.3.8 Version Control

- Version control, also known as source control, lies at the heart of every successful DevOps workflow. It helps DevOps teams stay organized, focused, and up to date with what members of the team are doing.
- Version control is also crucial for ensuring teams collaborate faster and easier to support frequent software releases.
- Version control is a central code repository,a place where developers contribute their code and track changes during the development process. Most version control systems allow team members to create branches of the main project. Branches are copies of the project's source code that individual developers can work on without modifying the original code.
- Git is a popular version control tool, in no time by downloading our free Git Commands.
- Each developer works on a separate branch of the same project, submitting code updates and running automated tests. Before the newly written code gets merged with the master branch, automation will create a build of the application that will ensure the new code is compatible with the existing code.
- If the build is successful, developers merge the new code with the master branch and deploy it to production or run other tests, depending on the workflow.



- With a robust version control system in place, DevOps teams can be confident that only error-free code is moving down the pipeline and getting deployed to production.

9.1.3.9 Collaboration

- The main idea behind DevOps is to establish trust among developers and operations. Dev and Ops teams need to communicate, share feedback, and collaborate throughout the entire development and deployment cycle.
- In such a setting, both groups are responsible for ensuring the application delivers on its promises. That requires continuously optimizing and improving the performance, costs, and delivery of services while keeping users satisfied.
- Creating this type of inclusive, collaborative environment also involves accepting a cultural shift within the company.
- To execute DevOps successfully, it's crucial to have the stakeholders and DevOps teams be on the same page, working together to deliver software solutions that bring real value to the company and its customers.
- DevOps requires the entire company to behave like a startup, continuously adjusting to market changes, and investing time and resources in features that attract more customers.
- DevOps revolves around specific technologies that help teams get the job done.
- DevOps is a culture. Building a devops culture requires an organization to work as a unified team, from Development and Operations to stakeholders and management.
- When transitioning to DevOps, remember that these principles are not set in stone. Organizations should implement DevOps methodologies based on their goals, workflows, resources, and the team's skill set.

9.1.4 Benefits of DevOps

- Companies that practice DevOps have reported significant benefits,
- Significantly shorter time to market,
- Improved customer satisfaction,
- Better product quality,

- More reliable releases,
- Improved productivity and efficiency,
- Increased ability to build the right product by fast experimentation.